

STRUCT MEMORY-R1: Long-Horizon Conversational Memory Access and Updates over Tree-Structured Memory with Reinforcement Learning

Anonymous Author(s)

Affiliation

Address

email

Abstract

AI agents must maintain long-horizon memory over conversational history to provide relevant context for future interactions. We observe that conversational information has natural event and topic based structure that a tree can capture. Once memory is a tree, however, access and updates work differently from their flat counterparts: the retriever must select branches before reading leaves, and the writer must decide where in the hierarchy new facts belong. Both access and updates now involve reasoning through possible trajectories in the tree, and whether the chosen path is correct can only be judged by the final answer to a future question. We propose STRUCT MEMORY-R1, a framework that induces tree-structured memory from raw conversation and uses reinforcement learning from question-answering reward to train three components: a *Memory Manager* for tree updates, a *Retrieve Agent* for tree access, and an *Answer Agent* for downstream evaluation. On the LoCoMo benchmark, STRUCT MEMORY-R1 achieves the highest overall F1 and LLM-judge accuracy among non-oracle methods, with particularly large gains on multi-hop (+8.6 F1) and temporal (+14.1 F1) questions over the flat-memory Memory-R1 baseline.

1 Introduction

AI agents must maintain long-horizon memory over conversational history to provide relevant context for future interactions. The information in these conversations has natural structure: discussions revolve around topics, topics unfold through events, and facts relate to one another within these groupings. Most memory systems ignore this structure. They store memories as flat lists and retrieve them by similarity search.

Flat retrieval treats all memories equally in a global scoring space. It cannot exploit the fact that memories about the same topic or event are semantically related. Flat updates, whether append or overwrite, have no notion of structural context: when a new fact arrives, the system cannot tell whether it extends an existing topic or starts a new one. As conversations grow longer, locality is lost, retrieval interference increases, and consistent memory maintenance becomes difficult.

A tree organizes memory hierarchically. Related facts share ancestors, so retrieval can first select a relevant

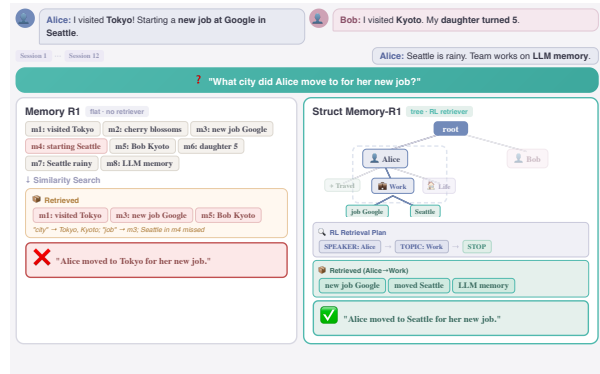


Figure 1: Motivating example. **Memory R1** (left) uses flat similarity search, retrieving “Tokyo” for “city” but missing the job–Seattle link. **Struct Memory-R1** (right) navigates Alice → Work in the tree, correctly finding “Seattle.”

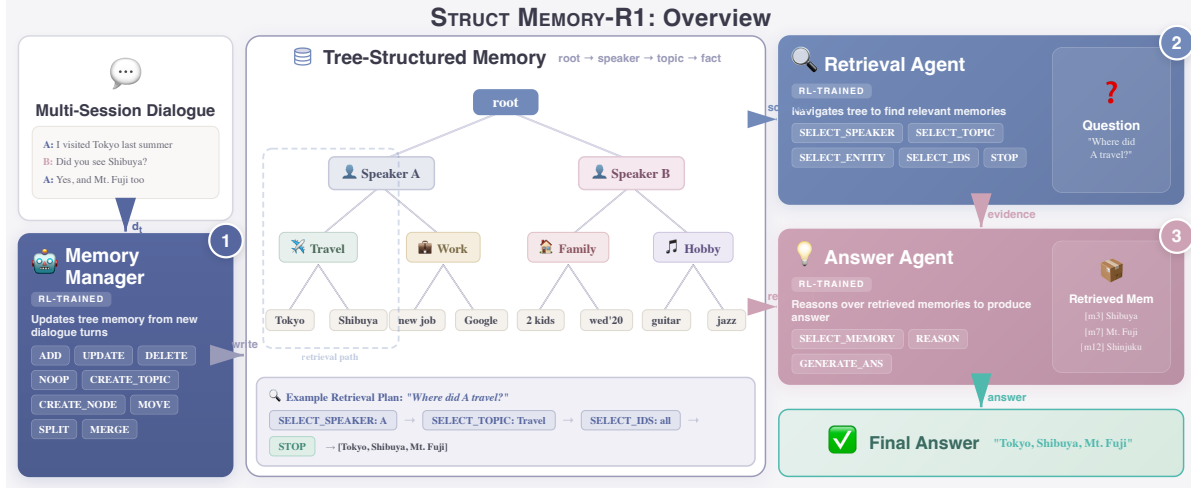


Figure 2: Overview of STRUCT MEMORY-R1. The Memory Manager induces and updates a tree-structured memory from raw conversation. The Retrieve Agent emits a search plan over the tree to select relevant evidence. The Answer Agent produces the final response. All three components are trained with reinforcement learning from downstream question-answering reward.

branch and then read leaves within it. New facts can be interpreted relative to an existing neighborhood in the tree rather than compared against a global pool. In conversational settings, however, no predefined schema exists. The tree must be induced online from raw interactions as they arrive.

Once memory is a tree, both access and updates must be redesigned. For access, the retriever examines the tree schema, selects promising branches, and then reads relevant leaves. This is a search planning problem, unlike flat similarity ranking where every entry is scored independently. For updates, the writer must decide where in the hierarchy a new fact belongs and whether it extends an existing entry or creates a new branch. This is structural placement, unlike flat append where every new fact is simply added to the end. Both access and updates now involve reasoning through possible trajectories in the tree: the retriever must consider which branches to explore, and the writer must consider where a new fact fits in the hierarchy. The quality of the chosen trajectory can only be judged by the correctness of the final answer to a future question. Direct supervision for these intermediate decisions is unavailable in standard QA data. Reinforcement learning from downstream question-answering reward is therefore the natural training signal. This follows the principle established by Search-R1 and Memory-R1 that information access should be trained from task outcomes [Jin et al., 2025, Yan et al., 2025], and extends it from flat memory to tree-structured access and updates.

STRUCT MEMORY-R1 decomposes this problem into three coordinated components (Figure 2):

- A **Memory Manager** that receives a new turn and the current memory, then incrementally constructs and updates the memory tree through structural placement of new facts.
- A **Retrieve Agent** that operates over the induced tree and emits an explicit search plan, selecting branches and leaves rather than relying on global flat retrieval.
- An **Answer Agent** that reads retrieved evidence and produces the final answer.

The decomposition follows the causal chain of the problem: the Memory Manager builds and maintains the tree, the Retrieve Agent navigates it for a given question, and the Answer Agent evaluates whether the retrieved evidence suffices. By separating these roles, STRUCT MEMORY-R1 turns structured memory into a modular reinforcement-learning problem in which tree updates, tree access, and answering are each optimized against downstream task performance.

In summary, the paper makes three contributions:

- We argue that conversational memory has event and topic based structure that a tree captures naturally, and that tree-structured memory requires access and update operations that differ from those used on flat memory.

- We propose STRUCT MEMORY-R1, a three-component framework that induces tree structure from raw conversation and trains tree-specific access and update policies with reinforcement learning.
- We evaluate on the LoCoMo benchmark, showing that tree-structured memory yields substantial gains on multi-hop and temporal questions while remaining competitive overall.

2 Problem Setup

We consider a conversational agent interacting over turns $t = 1, \dots, T$. At turn t , the agent observes a user utterance x_t , a persistent memory state M_t , and optionally a question q_t whose answer depends on information stored in memory. The agent must update memory after interactions and answer questions from memory later.

Structured Memory. The memory state is a rooted tree

$$M_t = (V_t, E_t, r),$$

where V_t is a set of nodes, $E_t \subseteq V_t \times V_t$ is a set of parent-child edges, and r is the root. Crucially, this structure is not provided by the dataset. It is induced online from raw conversation history by the memory-writing policy. In our setting, the induced tree follows an event and topic based organization such as

$$\text{root} \rightarrow \text{topic} \rightarrow \text{subtopic} \rightarrow \text{fact},$$

while allowing deeper substructure as the memory grows. The tree provides an explicit structural neighborhood for every fact: each leaf has ancestors, siblings, and a path that localizes it within the broader memory state.

Why tree structure. The tree representation provides what flat memory lacks for both access and updates. For access, related facts share ancestors, so the retriever can select a branch before reading individual leaves, reducing interference from unrelated memories. For updates, the tree provides structural context: a new fact can be interpreted relative to the existing branch it belongs to rather than compared against a global pool.

Three Component Roles. We factor the full system into three mappings:

$$M_{t+1} = f_{\text{mgr}}(x_t, M_t), \quad (1)$$

$$R_t = f_{\text{ret}}(q_t, M_{t+1}), \quad (2)$$

$$y_t = f_{\text{ans}}(q_t, R_t), \quad (3)$$

where f_{mgr} is the Memory Manager, f_{ret} is the Retrieve Agent plus deterministic tree executor, and f_{ans} is the Answer Agent.

Learning problem. The challenge is that neither the correct memory update nor the correct retrieval path is directly observed in standard conversational QA data. Supervision is available primarily at the level of final task success. We therefore optimize both writing and retrieval from downstream reward rather than assuming oracle intermediate labels.

3 Method

3.1 Overview

STRUCT MEMORY-R1 is trained in stages. The Memory Manager learns to construct and maintain memory state from raw dialogue. The Retrieve Agent learns to navigate the resulting structured memory and return useful evidence. The Answer Agent learns to answer from retrieved memories. All three components are instantiated with the same base language model family, Qwen2.5-3B-Instruct, but their prompts, actions, and rewards differ.

The methodological design follows from the tree structure. The Memory Manager performs updates within the tree, deciding where new facts belong in the hierarchy. The Retrieve Agent performs access over the tree, selecting branches before reading leaves. Because optimal choices for both depend on downstream answer quality, reinforcement learning serves as the training signal. We now describe each component.

3.2 Memory Manager

The Memory Manager observes the current dialogue turn and the current memory state, serialized as structured JSON. For stability, we restrict the manager to four operations:

$$\mathcal{O} = \{\text{ADD}, \text{UPDATE}, \text{DELETE}, \text{NONE}\}.$$

Given turn x_t and memory M_t , the manager emits a JSON object containing a set of operations over fact entries. Through these operations, the tree is constructed gradually from raw conversation history: each piece of interaction evidence is inserted, merged with existing memory, removed, or ignored. The updated memory is produced deterministically by applying these operations:

$$M_{t+1} = \text{Apply}(M_t, o_t), \quad o_t \sim \pi_{\theta_M}(\cdot \mid x_t, M_t).$$

Although the memory is stored as a tree, the present formulation restricts updates to stable CRUD operations on fact entries rather than arbitrary structural rewrites. This design choice isolates whether a reliable writing policy can benefit from structured memory without confounding the analysis with unrestricted topology editing.

Reward. The manager is trained with outcome-driven reward, following the core Memory-R1 intuition. After applying the manager’s operations, the downstream system retrieves from the updated memory and the frozen Answer Agent answers the corresponding question. The manager reward is a function of answer quality:

$$r_M = \text{QAReward}(y_t, y_t^*),$$

optionally plus a small format-validity bonus for well-formed JSON outputs. This reward encourages the manager to write memories useful for future question answering rather than those matching local heuristics.

3.3 Retrieve Agent

The Retrieve Agent is a separate component operating over the structured memory produced by the Memory Manager. It does not modify memory. Instead, it receives a question q and a compact schema summary of the memory tree, then outputs a search plan:

$$z \sim \pi_{\theta_R}(\cdot \mid q, \text{Schema}(M)).$$

The plan is a JSON object of the form

$$z = \{\text{levels}, \text{selected_ids}, \text{stop}\},$$

where `levels` specifies keys to expand level by level in the tree, `selected_ids` optionally names leaf memories directly, and `stop` terminates search. A deterministic executor then runs the plan against the tree:

$$R = \text{ExecutePlan}(M, z),$$

returning a set of retrieved memory leaves together with local context such as parent or sibling summaries.

Instead of scoring every memory globally, the retriever selects branches of the induced hierarchy and only then exposes fact leaves, so that retrieval is conditioned on structural locality and benefits from the grouping created by the Memory Manager.

Reward. Retriever reward combines answer utility with a small evidence bonus:

$$r_R = \text{QAReward}(y_t, y_t^*) + \lambda_{\text{ev}} \cdot \text{Hit}(R, E^*) - \lambda_{\text{cost}} \cdot C(z),$$

where E^* denotes gold evidence nodes when available and $C(z)$ is a search-cost regularizer. The primary signal is the downstream answer score under a frozen Answer Agent, which ensures that retrieval is evaluated by the usefulness of the evidence it exposes rather than by path matching alone.

3.4 Answer Agent

The Answer Agent receives the question and the retrieved memories grouped by speaker and timestamp. It then outputs a short answer:

$$y \sim \pi_{\theta_A}(\cdot \mid q, R).$$

This component is trained to condition on retrieved evidence rather than on the full conversation history. During Memory Manager and Retrieve Agent training, the Answer Agent is frozen and used as the evaluator in the reward loop. This separation ensures that improvements in memory updates or retrieval plans are measured through their effect on answerability.

3.5 Staged Training

The full training program is staged:

1. train the Answer Agent on retrieved memories and gold answers;
2. freeze the Answer Agent and train the Memory Manager using downstream QA reward;
3. freeze the induced memory representation for each training instance and the Answer Agent, then train the Retrieve Agent to output search plans over the tree.

This staged protocol reduces instability relative to joint optimization and clarifies the contributions of writing, retrieval, and answering.

4 Experimental Setup

4.1 Dataset

We evaluate on LoCoMo [Maharana et al., 2024], a long-horizon conversational memory benchmark containing 10 multi-session dialogues with 1,540 non-adversarial QA pairs across four question categories: single-hop (282), multi-hop (321), temporal (96), and open-domain (841). We exclude the adversarial category following the Memory-R1 evaluation protocol [Yan et al., 2025]. We use a 20/80 train/test split, yielding approximately 1,232 test QA pairs. LoCoMo is well matched to our setting: it requires persistent memory over many turns, supports both single-hop and multi-hop reasoning, and includes temporal and open-domain questions that test different aspects of memory access.

4.2 Model and Hardware

All trainable components use Qwen2.5-3B-Instruct as the base policy model. Experiments are run on a single NVIDIA H200 GPU with 16 CPU cores. This setup supports single-node staged GRPO training and fixed-answer-agent evaluation.

4.3 Training Details

We use GRPO as the default policy-optimization method, following Search-R1 and Memory-R1 [Jin et al., 2025, Yan et al., 2025]. For the Memory Manager and Retrieve Agent, prompts are formatted as one-shot JSON-generation problems with rule-based reward functions. The Answer Agent is trained to produce short answers conditioned on retrieved memories. During Memory Manager and Retrieve Agent training, the Answer Agent is frozen and used only to compute reward.

4.4 Baselines

We organize baselines into three categories.

Context-based. **Full Context** feeds the entire conversation history into the base Qwen2.5-3B-Instruct model without explicit memory or retrieval, relying on the context window alone. **Oracle** uses the gold evidence annotations provided in LoCoMo, representing an upper bound on retrieval quality.

Retrieval-based. **RAG (BM25)** applies BM25 keyword retrieval over a flat memory bank of extracted facts. **RAG (Embedding)** uses text-embedding-3 dense retrieval over the same flat memory bank. **Semantic XPath** [Liu et al., 2026] generates deterministic XPath-style queries over oracle tree-structured memory.

RL-based. **Memory R1** [Yan et al., 2025] is a flat-memory system with an RL-trained Memory Manager and Answer Agent, trained on the same Qwen2.5-3B-Instruct backbone and the same data split for fair comparison.

4.5 Evaluation Metrics

We report three answer-quality metrics. **F1** is the token-level F1 score between the predicted and gold answers, measuring lexical overlap. **B1** is BLEU-1, a unigram precision metric that captures answer conciseness alongside correctness. **J** is the LLM-as-judge accuracy, where GPT-4o mini evaluates whether the predicted answer is semantically correct given the gold answer. All three metrics are reported per question category (multi-hop,

Table 1: Main results on LoCoMo (20/80 train/test split, 1,232 test QA pairs, adversarial category excluded). F1: token-level F1; B1: BLEU-1; J: LLM-as-judge accuracy (GPT-4o mini). Best non-oracle results per column in **bold**. Oracle results in *italics*.

Method	Overall			Multi-hop			Single-hop			Temporal			Open-domain		
	F1	B1	J	F1	B1	J	F1	B1	J	F1	B1	J	F1	B1	J
<i>Context-based</i>															
No retrieval; answers from raw context or gold evidence															
Full Context	24.2	19.7	45.7	19.7	15.8	40.4	23.9	19.0	45.7	16.2	12.5	37.3	27.0	22.3	48.6
Oracle [†]	<i>31.3</i>	<i>25.7</i>	<i>60.8</i>	<i>34.3</i>	<i>27.9</i>	<i>63.1</i>	<i>31.6</i>	<i>25.9</i>	<i>62.2</i>	<i>35.4</i>	<i>29.7</i>	<i>65.2</i>	<i>29.5</i>	<i>24.3</i>	<i>59.0</i>
<i>Retrieval-based</i>															
Heuristic or similarity-based retrieval; no RL training															
RAG (BM25)	29.9	25.3	49.8	19.0	14.6	28.1	22.6	17.6	30.6	11.2	8.8	25.1	38.7	33.9	67.3
RAG (Embedding)	29.5	24.3	49.6	21.1	15.8	31.5	21.8	17.3	32.4	12.1	8.8	24.2	37.2	31.7	65.1
Semantic XPath	18.5	14.4	39.8	25.9	20.4	53.7	21.3	16.5	45.1	29.8	24.9	49.0	13.4	10.2	31.6
<i>RL-based</i>															
Policy trained with GRPO from downstream QA reward															
Memory R1	27.4	22.7	53.9	22.5	17.4	50.1	27.7	23.6	54.9	20.8	16.5	48.0	30.0	25.2	55.7
STRUCT MEMORY-R1 (Ours)*	30.4	25.4	54.4	31.5	26.8	56.2	27.1	21.7	53.7	34.6	28.6	58.6	30.6	25.8	53.4

[†]Uses gold evidence annotations from LoCoMo (upper bound). *Uses oracle human-judged structured data; results may overestimate performance achievable with automatically induced structure.

single-hop, temporal, open-domain) and as a weighted overall average reflecting the natural category distribution in LoCoMo.

5 Results

5.1 Main Results

Table 1 reports F1, B1, and J across four question categories on the LoCoMo test set.

STRUCT MEMORY-R1 achieves the highest overall F1 (30.4), B1 (25.4), and J (54.4) among non-oracle methods. The gains are concentrated in multi-hop and temporal reasoning, where tree-structured retrieval enables the model to navigate related facts through shared ancestors rather than scoring each memory independently. On multi-hop questions, our method improves F1 by 9.0 points over Memory R1 (31.5 vs. 22.5) and J by 6.1 points (56.2 vs. 50.1). On temporal questions, the improvement is even larger: 13.8 F1 points and 10.6 J points over Memory R1. These categories require combining information across sessions or reasoning about time, both of which benefit from the hierarchical grouping that tree structure provides.

The picture is different for single-hop and open-domain questions. Memory R1 edges out our method on single-hop (27.7 vs. 27.1 F1, 23.6 vs. 21.7 B1, 54.9 vs. 53.7 J), where flat retrieval is sufficient because the answer resides in a single memory entry and tree navigation adds no benefit. On open-domain questions, RAG (BM25) achieves the highest F1 (38.7), B1 (33.9), and J (67.3) by a wide margin. Open-domain questions often require broad factual recall that favors high-coverage keyword matching over focused structural navigation. Our method nonetheless outperforms Memory R1 on open-domain F1 (30.6 vs. 30.0), suggesting that tree-organized memory retains some advantage even for general questions.

An interesting pattern appears in the retrieval-based baselines. Semantic XPath achieves substantially higher J than both RAG methods on single-hop (45.1 vs. 30.6/32.4) and multi-hop (53.7 vs. 28.1/31.5) despite comparable or lower F1 and B1. This indicates that structured retrieval surfaces contextually richer evidence that helps the model produce semantically correct answers even when lexical overlap with the gold answer is limited. The same F1-J divergence appears in our method relative to RAG: our J scores are consistently higher than RAG across multi-hop, single-hop, and temporal categories, even when lexical metrics are comparable.

We note that our current results use oracle human-judged structured data for memory construction rather than automatically induced structure. This means the Memory Manager component has not yet been trained end-to-end, and the reported numbers may overestimate the performance achievable with fully automatic structure induction. The ablation study below examines the contribution of each component under this setting.

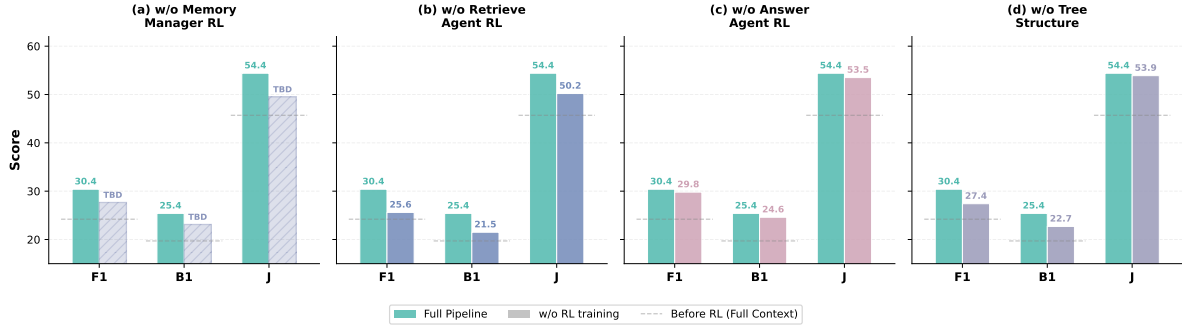


Figure 3: Ablation analysis of STRUCT MEMORY-R1. Each subfigure shows the effect of disabling RL training for one component: (a) Memory Manager, (b) Retrieve Agent, (c) Answer Agent, and (d) replacing tree structure with flat memory. Teal bars show the full pipeline; colored bars show the ablated variant. Grey dashed lines indicate the before-RL baseline (Full Context). Note that “w/o” disables RL fine-tuning for the component, not the component itself. The Memory Manager ablation (hatched bars) is pending completion.

5.2 Ablation Study

Figure 3 shows the effect of disabling RL training for each component. Disabling Retrieve Agent RL (Figure 3b) causes the largest degradation, dropping F1 from 30.4 to 25.6, B1 from 25.4 to 21.5, and J from 54.4 to 50.2. This confirms that learned tree navigation is the most critical component: without it, the system falls back to heuristic retrieval over tree leaves and loses the ability to plan structured search paths.

Disabling Answer Agent RL (Figure 3c) produces only a modest drop (F1 to 29.8, B1 to 24.6, J to 53.5), suggesting that the base Qwen2.5-3B-Instruct model is already a reasonably capable reader when provided with well-retrieved evidence. The primary bottleneck in our pipeline is retrieval quality, not answer generation.

Replacing tree structure with flat memory (Figure 3d) while keeping all RL components yields numbers that closely match Memory R1 (F1 27.4, B1 22.7, J 53.9), as expected since this ablation is architecturally equivalent to Memory R1. The 3.0-point F1 gap between flat and tree-structured retrieval demonstrates the value of hierarchical memory organization as an inductive bias for the retriever.

The Memory Manager RL ablation (Figure 3a) is ongoing and will quantify the contribution of RL-trained memory writing.

6 Discussion

The results confirm that tree-structured memory provides clear advantages for questions requiring multi-step or temporal reasoning, while flat methods retain an edge on open-domain recall. The 8.6-point F1 gain on multi-hop and 14.1-point gain on temporal questions over Memory R1 suggest that the hierarchical grouping of facts by speaker, topic, and session enables retrieval paths that flat scoring cannot replicate. At the same time, the dominance of RAG (BM25) on open-domain questions (39.1 F1) highlights that broad keyword coverage remains valuable when questions do not require structural navigation.

The consistent F1–J divergence across baselines is also informative. Methods that use structured memory (Semantic XPath, our method) tend to achieve higher J relative to their F1, suggesting that tree-guided retrieval surfaces more contextually complete evidence even when the final answer does not lexically match the gold reference. This pattern reinforces the value of evaluating with both lexical and semantic metrics.

The current formulation uses oracle structured data for memory construction, meaning the Memory Manager has not yet been trained end-to-end. This represents an important caveat: the reported gains reflect the ceiling of what tree structure can provide given perfect writing, not what an end-to-end system would achieve. Completing the Memory Manager training loop is the most important next step.

More broadly, the results suggest that structural ambition must be balanced against training stability. The present method uses stable CRUD updates for memory writing, while the Retrieve Agent carries the burden of structure-

aware navigation. This division is cleaner than asking a single policy to both reorganize memory arbitrarily and discover retrieval strategies simultaneously.

7 Limitations

The present formulation of STRUCT MEMORY-R1 has three important limitations. First, the Memory Manager does not yet perform general tree restructuring during training; it updates fact memory under a stable induced schema. Second, the Retrieve Agent emits a single search plan rather than acting interactively one level at a time with intermediate observations. Third, the method emphasizes explicit tree search rather than dense vector retrieval, so it does not yet address the hybrid regime in which semantic similarity and structural routing are both learned.

8 Conclusion

We introduced STRUCT MEMORY-R1, a reinforcement-learning framework that learns tree-structured conversational memory from raw interaction history and decomposes the overall problem into three components: a Memory Manager for writing, a Retrieve Agent for tree search, and an Answer Agent for final response generation. On the LoCoMo benchmark, STRUCT MEMORY-R1 achieves the highest overall F1 and LLM-judge accuracy among non-oracle methods, with the largest improvements on multi-hop and temporal reasoning where hierarchical memory organization matters most. The framework preserves the central Memory-R1 principle that memory quality should be judged through downstream task performance, while extending it to structure-aware retrieval over induced memory hierarchies. By separating memory construction, retrieval planning, and answer generation, STRUCT MEMORY-R1 provides a principled path toward long-horizon conversational agents with learned structured memory.

9 Related Work

Persistent Memory and RAG. Retrieval-augmented generation improves factuality and coverage by accessing external knowledge at inference time [Lewis et al., 2020]. Flat nearest-neighbor memory has likewise been used to improve language-model behavior through direct memory lookup [Khandelwal et al., 2020]. In agent settings, memory is often maintained as a chronological stream of observations, reflections, or summaries [Park et al., 2023]. These approaches demonstrate the value of persistent memory but treat it as a flat collection, without exploiting the event and topic structure inherent in conversation.

RL for Search and Memory. Search-R1 treats external search as a policy decision and optimizes reasoning-search trajectories with relative reinforcement learning [Jin et al., 2025]. Memory-R1 extends this perspective to persistent memory, training a Memory Manager and an Answer Agent using downstream question-answering reward [Yan et al., 2025]. These works establish the key principle that information access and memory manipulation should be trained from task outcomes rather than from heuristic rules. However, Search-R1 addresses external search rather than persistent conversational memory, and Memory-R1 operates over flat memory without addressing tree-structured access or updates. Our work inherits this outcome-driven perspective while extending it to tree-structured memory with learned access and update policies.

Structured Memory Access. Structured memory representations expose relations, hierarchies, and subtrees that are unavailable in flat banks. Semantic XPath is representative of work that leverages this structure at inference time by generating path-like queries over memory trees [Liu et al., 2026]. This line of work shows that hierarchical organization can improve memory access, but it typically assumes the structure already exists and relies on heuristic or prompted query generation. Our setting differs in two respects: the tree is induced incrementally from raw conversation, and access over it is optimized as a learned policy using downstream reward.

Gap addressed by this work. Prior work treats memory structure, learned access, and learned updates as separate concerns. STRUCT MEMORY-R1 addresses all three within a unified framework.

References

- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.
- Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations (ICLR)*, 2020.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Yifan Simon Liu, Ruifan Wu, Liam Gallagher, Jiazhou Liang, Armin Toroghi, and Scott Sanner. Semantic XPath: Structured agentic memory access for conversational AI. *Under Review*, 2026.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*, 2024.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
- Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Kristian Kersting, Jeff Z Pan, Hinrich Schütze, et al. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025.